Empirical Evaluation of Small Language Models on Edge Computing Architecture

Golam Masum Basar
Independent Research, Gollama Project
June 13, 2026

Abstract

This paper presents an empirical evaluation of four state-of-the-art small language models (SLMs) operating within a fully isolated, offline edge environment on Apple M2 hardware with 8 GB of unified memory. We assess three core dimensions: generation throughput and latency, mathematical reasoning accuracy, and structured JSON schema extraction stability across varying temperature configurations. Our findings reveal significant divergence between dense and reasoning-specialized architectures, and demonstrate that schema extraction reliability is highly non-linear under entropy variation. Practical deployment criteria for localized microservice architectures on constrained edge hardware are derived from these results.

1. Introduction

This research evaluates edge AI optimization across localized model architectures. To establish genuine computational execution constraints, all empirical testing was conducted natively on a MacBook Air M2 configured with 8 GB of unified RAM. We assess processing speed, mathematical reasoning, and structural schema parsing reliability across four state-of-the-art small language models operating within an isolated offline environment.

The four models evaluated are: **Gemma 3 1B**, **Llama 3.2 1B**, **DeepSeek R1 1.5B**, and **Qwen 2.5 1.5B**. All models were served locally via Ollama and queried through a custom FastAPI wrapper (Gollama). Raw telemetry logs were collected directly from the Gollama benchmarking endpoint.

2. Speed and Efficiency Performance Benchmarking

Benchmark Prompt 1

"Write a concise 4-sentence paragraph explaining why running an AI model locally is better than using a cloud API."

Table 1: Computational Throughput Baseline

Model	TTFT (ms)	Latency (s)	Tokens	Throughput (tok/s)
Gemma 3 1B	1998.12	4.89	73	25.24
Llama 3.2 1B	2725.86	6.54	106	27.79
DeepSeek R1 1.5B	2401.13	30.01	541	19.60
Qwen 2.5 1.5B	2595.08	7.19	91	19.79

Benchmark Prompt 2

"A farmer has chickens and rabbits. There are 35 heads and 94 legs in total. Step by step, calculate how many chickens and rabbits the farmer has."

Table 2: Mathematical Reasoning Accuracy

Model	TTFT (ms)	Latency (s)	Tokens	Accuracy Outcome
Gemma 3 1B	2088.60	14.95	262	Correct (23 chickens, 12 rabbits)
Llama 3.2 1B	2657.58	23.67	378	Failed (mathematical hallucination)
DeepSeek R1 1.5B	2376.52	33.27	614	Correct (23 chickens, 12 rabbits)
Qwen 2.5 1.5B	2620.02	25.68	441	Correct (23 chickens, 12 rabbits)

Performance Benchmarking Discussion

The metrics reveal a distinct divergence between standard dense architectures and reasoning-specialized setups. Dense models achieve superior throughput velocities — up to 27.79 tok/s — with sub-3-second time-to-first-token. However, Llama 3.2 1B demonstrates severe analytical vulnerability, failing entirely during algebraic verification. DeepSeek R1 1.5B maintains correct mathematical resolution but incurs a 300% latency overhead attributable to its internal chain-of-thought generation mechanism.

3. Temperature Effects on Structured Schema Extraction

Extraction Prompt 1 (Profile 1)

"Hi, I am Jessica Sterling. I am a software engineer with 5 years of experience. My day-to-day stack consists of Python, FastAPI, and PostgreSQL. I also occasionally work with TypeScript and React for building dynamic interfaces."

Table 3: Gemma 3 1B — Schema Extraction Integrity (Profile 1)

Temp. Band	Schema Status	Array Coverage	Observed Behaviour
0.0 – 1.5	Pass	Complete: [Python, FastAPI, PostgreSQL, TypeScript, React]	Consistent structural stability across all entropy bounds

Table 4: Llama 3.2 1B — Schema Extraction Integrity (Profile 1)

Temp. Band	Schema Status	Array Coverage	Observed Behaviour
0.0 – 0.5	Pass	Complete: [FastAPI, PostgreSQL, TypeScript, React]	Standard operational execution
0.6 – 0.9	Partial	Incomplete: [FastAPI, PostgreSQL]	Silent truncation; secondary skills dropped
1.0 – 1.5	Pass	Complete: [FastAPI, PostgreSQL, TypeScript, React]	Structural syntax recovers under high entropy

Table 5: DeepSeek R1 1.5B — Schema Extraction Integrity (Profile 1)

Temp. Band	Schema Status	Array Coverage	Observed Behaviour
0.0 – 1.5	Pass	Complete: [FastAPI, PostgreSQL, TypeScript, React]	Full schema compliance; elevated latency (14–17 s)

Table 6: Qwen 2.5 1.5B — Schema Extraction Integrity (Profile 1)

Temp. Band	Schema Status	Array Coverage	Observed Behaviour
0.0	Partial	Incomplete: [FastAPI, PostgreSQL]	Array truncation under greedy decoding
0.6	Pass	Complete: [FastAPI, PostgreSQL, TypeScript, React]	Structural normalization achieved
1.5	Pass	Maximum: [Python, FastAPI, PostgreSQL, TypeScript, React]	Entropy expansion enables full field coverage

Extraction Prompt 2 (Profile 2)

"The project lead, Thomas Müller, has been engineering distributed backends for a decade. He specialises in React, Node.js, and MongoDB, but also handles Docker deployments."

Table 7: Gemma 3 1B — Contextual Schema Parsing (Profile 2)

Temp. Band	Language Field	Exp.	Observed Behaviour
0.0–0.4, 0.6, 0.9, 1.5	JavaScript (inferred)	10	Correct implicit language abstraction across wide entropy range
0.5, 0.7, 0.8, 1.0	React (literal)	10	Attention drop clips Docker from skills array

Table 8: Llama 3.2 1B — Contextual Schema Parsing (Profile 2)

Temp. Band	Language Field	Exp.	Observed Behaviour
0.0, 0.2–0.5	JavaScript (inferred)	10	Docker skill node truncated
0.1, 0.6	JavaScript (inferred)	10	Optimal band; full context accuracy
0.8	`JavaScript, primarily Node.js}`	10	Syntax token leakage into text field
1.5	`English (Programmers)...}`	10	Semantic disintegration of parameter values

Table 9: DeepSeek R1 1.5B — Contextual Schema Parsing (Profile 2)

Temp. Band	Language Field	Exp.	Observed Behaviour
0.0–0.5	React, Node.js, MongoDB	10	Flat string clustering; no abstract language deduction
1.0	React, Node.js, MongoDB	10	Conversational descriptors leak into list items
1.5	React, Node.js, MongoDB	10	Array field collapsed into single block

Table 10: Qwen 2.5 1.5B — Contextual Schema Parsing (Profile 2)

Temp. Band	Language Field	Exp.	Observed Behaviour
0.0–0.4, 0.6, 1.0–1.5	JavaScript (inferred)	10	Consistent correctness at low and high entropy bounds
0.5	React, Node.js, MongoDB	10	Frameworks omitted from skills array
0.7, 0.8	Multi-string descriptive text	10	Structural shift from keywords to phrase descriptions

Extraction Discussion

Structured schema extraction results confirm that model capability under entropy variation is highly non-linear. While all models cleanly handle straightforward type mappings (e.g., numeric string to integer), attention boundaries fluctuate considerably. Llama architectures degrade under elevated temperature, exhibiting semantic collapse and token syntax leakage into field values. Qwen configurations manifest localized mid-range instability where accuracy decreases before recovering at higher entropy bounds. Gemma 3 1B demonstrates the most consistent schema adherence across the full temperature spectrum, making it the most reliable backbone for deterministic extraction pipelines on constrained hardware.

4. Conclusion

These empirical findings establish practical deployment criteria for localized microservices on constrained edge systems. For latency-sensitive applications, dense architectures such as Gemma 3 1B and Llama 3.2 1B offer superior throughput, though Llama demonstrates unacceptable reasoning failures. DeepSeek R1 1.5B provides reliable mathematical accuracy at the cost of significant latency overhead. For structured data extraction, temperature should be fixed at or near zero, and Gemma 3 1B is recommended as the primary extraction backbone due to its consistent schema compliance across all tested entropy configurations. System architects must carefully match throughput requirements against model-specific structural stability characteristics to guarantee operational reliability on production edge deployments.